

CS687 EndSem Solutions

Krishna Kumar Bais

Roll No.: 241110038

Question 1:

Show that if there is a constant c such that for the sequence $X \in \Sigma^\infty$, we have

$$C(X[0 \dots (n-1)] \mid n) < c \text{ for all } n,$$

then X is computable.

Solution:

-> Goal:

We need to **prove that X is computable**, i.e., there exists a total computable function $f : \mathbb{N} \rightarrow \{0, 1\}$ such that $f(n) = X(n)$, the n th bit of X .

-> Understand the assumption:

We are given that:

$$\forall n \in \mathbb{N}, \quad C(X[0 \dots (n-1)] \mid n) < c,$$

This means: for each n , there exists a **program** p_n of length $< c$ such that when run on input n , it outputs $X[0 \dots (n-1)]$.

So:

- $U(p_n, n) = X[0 \dots (n-1)]$,
- where U is a fixed universal Turing machine.

-> There are finitely many such programs:

Since $C(\cdot \mid n) < c$, and program size $< c$, there are at most:

$$N = 2^c - 1$$

distinct programs that could serve as p_n for **any** n . Let the full list of such programs be:

$$\mathcal{P} = \{p_1, p_2, \dots, p_N\}$$

Each p_i is a candidate decoder for $X[0..n-1]$, when given n .

-> **Fix one program that works for all large n :**

Each $p_i \in \mathcal{P}$ defines a **partial function** $f_i(n) = U(p_i, n)$, potentially defined for infinitely many n .

We know that for each n , **at least one** p_i satisfies:

$$U(p_i, n) = X[0 \dots (n-1)]$$

But to conclude **computability**, we need **one single program** p_i that works for **all sufficiently large n** .

-> **Pigeonhole principle $\rightarrow$ one $p^* \in \mathcal{P}$ works infinitely often:**

- There are only finitely many p_i .
- So, by the pigeonhole principle, **some program** $p^* \in \mathcal{P}$ must work for **infinitely many n** .

Let:

$$S = \{n \in \mathbb{N} : U(p^*, n) = X[0 \dots (n-1)]\}$$

Then $S \subseteq \mathbb{N}$ is infinite.

-> **Show that S must be cofinite (i.e., missing only finitely many n):**

Let us suppose, for contradiction, that S is **not cofinite**; i.e., $\mathbb{N} \setminus S$ is infinite.

Then infinitely many n use programs **different** from p^* . But again, the number of programs is finite, so some other p_i would also be used infinitely often.

This process leads to an infinite sequence of switches between candidate programs — which would imply **infinitely many distinct infinite sequences** are being reconstructed by programs of size $< c$.

But there are **only finitely many** such programs, and each program p defines at most **one infinite sequence** (since $U(p, n)$ must agree with $X[0 \dots (n-1)]$ for all large n).

Thus, we reach a contradiction unless **one** p^* outputs $X[0..n-1]$ for **all sufficiently large** n .

So there exists a **fixed program** p^* such that:

$$\exists N_0 \forall n \geq N_0, \quad U(p^*, n) = X[0 \dots (n-1)]$$

-> **Use p^* to define a total computable function $f(n) = X(n)$:**

Define the function $f : \mathbb{N} \rightarrow \{0, 1\}$ as:

- For $n < N_0$, hardcode the value of $X(n)$.
- For $n \geq N_0$:
 1. Compute $X[0 \dots (n-1)] = U(p^*, n)$
 2. Return $X(n) = \text{nth bit of } U(p^*, n)$

Since:

- p^* is fixed and finite,
 - U is universal (and total on this domain),
 - and the hardcoded part is finite,
- $\Rightarrow f(n)$ is total and computable.

Conclusion:

$$X(n) = f(n), \quad \text{where } f \text{ is total computable}$$

Hence, the infinite sequence X is computable.

If there is a constant c such that for all n , $C(X[0 \dots (n-1)] \mid n) < c$, then X is computable.

Question 2:

Show that if for any infinite binary sequence X :

$$\sum_{n \in \mathbb{N}} 2^{n-K(X[0 \dots (n-1)])} < \infty \quad (1)$$

then:

$$\lim_{n \rightarrow \infty} K(X[0 \dots (n-1)]) - n = \infty \quad (2)$$

Solution:

-> Goal:

We need to prove that if the sum in (1) is finite, then the quantity $K(X[0 \dots (n-1)]) - n$ grows unbounded — i.e., the prefix complexities grow **strictly faster than linear**, asymptotically.

-> Intuition and Structure:

We are asked to show that the prefix complexity $K(X[0 \dots (n-1)])$ must eventually grow faster than n , under the assumption that the sum

$$\sum_{n \in \mathbb{N}} 2^{n-K(X[0 \dots (n-1)])}$$

is finite.

Let us define:

$$\bullet \ s_n := K(X[0 \dots (n-1)])$$

So (1) becomes:

$$\sum_{n \in \mathbb{N}} 2^{n-s_n} < \infty$$

We want to conclude that:

$$\lim_{n \rightarrow \infty} (s_n - n) = \infty \quad (\text{i.e., } s_n \gg n)$$

If the difference $s_n - n$ were bounded, say $s_n \leq n + c$, then the sum becomes:

$$\sum_n 2^{n-s_n} \geq \sum_n 2^{n-(n+c)} = \sum_n 2^{-c} = \infty$$

— contradiction.

-> Assumption (for contradiction):

Suppose the limit in (2) does **not** go to ∞ . Then there exists a constant $c \in \mathbb{N}$ and an **infinite** subsequence $\{n_k\}$ such that:

$$K(X[0 \dots (n_k - 1)]) \leq n_k + c \quad \text{for infinitely many } k$$

Then:

$$2^{n_k - K(X[0 \dots (n_k - 1)])} \geq 2^{n_k - (n_k + c)} = 2^{-c}$$

Therefore, for infinitely many n_k :

$$2^{n_k - K(X[0 \dots (n_k - 1)])} \geq 2^{-c}$$

So the sum:

$$\sum_n 2^{n - K(X[0 \dots (n - 1)])}$$

has infinitely many terms each $\geq 2^{-c}$, hence:

$$\sum_n 2^{n - K(X[0 \dots (n - 1)])} = \infty$$

This **contradicts the given assumption** (1) that the sum is finite.

Conclusion:

Therefore, our assumption must be false, and the only possibility is:

$$\lim_{n \rightarrow \infty} K(X[0 \dots (n - 1)]) - n = \infty \quad \text{(proved)}$$

If $\sum_{n \in \mathbb{N}} 2^{n - K(X[0 \dots (n - 1)])} < \infty$ for an infinite binary sequence X , then:

$$\lim_{n \rightarrow \infty} K(X[0 \dots (n - 1)]) - n = \infty$$

Question 3:

Suppose $d : \Sigma^* \rightarrow [0, \infty)$ is a **lower semicomputable martingale**. Let X be a sequence on which d **succeeds**. Then show that X has **infinitely many prefixes with low self-delimiting Kolmogorov complexity**.

Solution:

-> Goal:

We need to prove:

If $\limsup_{n \rightarrow \infty} d(X[0 \dots n-1]) = \infty$, then

$\exists$ infinitely many $\sigma \prec X$ such that $K(\sigma) \leq |\sigma| - \log d(\sigma) + O(1)$

-> Intuition:

A **lower semicomputable martingale** d is a computable betting strategy. If it **succeeds** on an infinite binary sequence X , that means $d(X[0..n]) \rightarrow \infty$ along a subsequence — i.e., it wins unbounded money by betting on X .

Such sequences must then **fail randomness tests** (they are not Martin-Löf random). One way to witness failure of randomness is that **some prefixes of the sequence have unexpectedly low prefix-free Kolmogorov complexity** — they are "too simple to be random".

We'll formalize this using **Kraft's inequality**, **code construction**, and known theorems (e.g., from class notes and Li & Vitányi) connecting martingale success to complexity bounds..

-> Definitions:

Let:

- Σ^* = set of finite binary strings.
- $d : \Sigma^* \rightarrow \mathbb{R}_{\geq 0}$ is a **lower semicomputable martingale**:
 - $d(\sigma) = \frac{1}{2}(d(\sigma 0) + d(\sigma 1))$
 - Computably approximable from below.
- d **succeeds** on $X \in \Sigma^\infty$:

$$\limsup_{n \rightarrow \infty} d(X[0..n-1]) = \infty$$

Let $\sigma_n = X[0..n-1]$. We aim to show that:

$$\text{For infinitely many } n, \quad K(\sigma_n) \leq |\sigma_n| - \log d(\sigma_n) + O(1)$$

-> **Use the Kraft Inequality via Ville's Lemma:**

We define a set of strings where the martingale exceeds some fixed capital:

Let $c \in \mathbb{N}$. Define:

$$S_c = \{\sigma \in \Sigma^* : d(\sigma) \geq 2^c\}$$

Ville's inequality gives:

$$\sum_{\sigma \in S_c} 2^{-|\sigma|} \leq 2^{-c}$$

This allows us to define a **prefix-free code** for strings in S_c , with lengths:

$$\ell(\sigma) \approx |\sigma| - \log d(\sigma) + O(1)$$

To explain this: Since $\sum_{\sigma} 2^{-\ell(\sigma)} \leq 1$, such codes obey **Kraft's inequality** and can be interpreted as the lengths of programs (i.e., descriptions) in a prefix code.

-> **Prefix Complexity from Code Lengths:**

Using the fact that the prefix-free Kolmogorov complexity $K(\sigma)$ is upper-bounded by the length of a prefix code for σ :

$$K(\sigma) \leq \ell(\sigma) + O(1) \leq |\sigma| - \log d(\sigma) + O(1)$$

This bound is **tight and valid** for all $\sigma \in S_c$.

-> **Apply to X :**

Since d succeeds on X , for **infinitely many** n , we have:

$$d(X[0..n-1]) \geq 2^c \quad \text{for arbitrarily large } c$$

So:

- For infinitely many n , $\sigma_n \in S_c$ for increasing c
- For such σ_n , we have:

$$K(\sigma_n) \leq |\sigma_n| - \log d(\sigma_n) + O(1)$$

This proves that X has **infinitely many prefixes** with **lower-than-expected Kolmogorov complexity**, compared to the length of the prefix.

Conclusion:

If d is a lower semicomputable martingale and d succeeds on an infinite binary sequence X , then:

There are infinitely many $\sigma \prec X$ such that: $K(\sigma) \leq |\sigma| - \log d(\sigma) + O(1)$

i.e., X has infinitely many prefixes with **low prefix-free Kolmogorov complexity**.

Problem 4:

Show that the following inequality holds for all strings x, y and z of length n .

$$2C(x, y, z) \leq C(x, y) + C(x, z) + C(y, z) + O(\log n)$$

Solution:

-> Preliminaries and Definitions:

Let $C(u)$ denote the plain Kolmogorov complexity of string u .

Let $C(u, v)$ denote the complexity of the pair (u, v) , i.e., the length of the shortest program that outputs both u and v .

Standard facts (used without proof here):

1. **Chain rule:**

$$C(u, v) = C(u) + C(v|u) + O(\log n)$$

2. **Triple chain rule:**

$$C(u, v, w) = C(u, v) + C(w|u, v) + O(\log n)$$

3. **Symmetry of information:**

$$C(v|u) + C(u) = C(u, v) + O(\log n)$$

4. For any strings u, v, w :

$$C(v|u, w) + C(w|u, v) \leq C(v, w|u) + O(\log n)$$

-> Step-by-step Proof:

Step 1: Write $C(x, y, z)$ in two ways

Use the triple chain rule:

$$C(x, y, z) = C(x, y) + C(z|x, y) + O(\log n) \tag{1}$$

$$C(x, y, z) = C(x, z) + C(y|x, z) + O(\log n) \tag{2}$$

Step 2: Add the two expressions

Add equations (1) and (2):

$$2C(x, y, z) = C(x, y) + C(x, z) + C(z|x, y) + C(y|x, z) + O(\log n) \tag{3}$$

Step 3: Bound the conditional complexities

Use the inequality:

$$C(z|x, y) + C(y|x, z) \leq C(y, z|x) + O(\log n) \quad (4)$$

Then add this to (3):

$$2C(x, y, z) \leq C(x, y) + C(x, z) + C(y, z|x) + O(\log n) \quad (5)$$

Now use the chain rule again:

$$C(y, z|x) \leq C(y, z) - C(x) + O(\log n) \quad (6)$$

But this doesn't simplify directly. Instead, we use the looser inequality:

$$C(y, z|x) \leq C(y, z) + O(\log n)$$

Substitute into (5):

$$2C(x, y, z) \leq C(x, y) + C(x, z) + C(y, z) + O(\log n) \quad (7)$$

Conclusion:

We have derived:

$$2C(x, y, z) \leq C(x, y) + C(x, z) + C(y, z) + O(\log n)$$

which holds for all strings x, y, z of length at most n , and the hidden constant in $O(\log n)$ is independent of the strings.